

**Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

**Project Report**

| <b>Sr.<br/>No.</b> | <b>INDEX</b>                        |  |
|--------------------|-------------------------------------|--|
| <b>1.</b>          | <b>Abstract</b>                     |  |
| <b>2.</b>          | <b>Introduction</b>                 |  |
| <b>3.</b>          | <b>Objectives</b>                   |  |
| <b>4.</b>          | <b>Software and Tools Used</b>      |  |
| <b>5.</b>          | <b>Design Description</b>           |  |
| <b>6.</b>          | <b>RTL Code</b>                     |  |
| <b>7.</b>          | <b>Testbench</b>                    |  |
| <b>8.</b>          | <b>RTL Simulation</b>               |  |
| <b>9.</b>          | <b>Synthesis</b>                    |  |
| <b>10.</b>         | <b>Gate-Level Simulation</b>        |  |
| <b>11.</b>         | <b>Static Timing Analysis (STA)</b> |  |
| <b>12.</b>         | <b>Results</b>                      |  |
| <b>13.</b>         | <b>Applications</b>                 |  |
| <b>14.</b>         | <b>Conclusion</b>                   |  |
| <b>15.</b>         | <b>References</b>                   |  |

# **Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

## **Abstract**

This project demonstrates the design, synthesis, and gate-level simulation of a 4-bit synchronous counter using Verilog HDL. The design was developed and verified using Xilinx Vivado. The RTL description was synthesized into a gate-level netlist, followed by post-synthesis functional simulation to verify correct operation. Static Timing Analysis (STA) was also performed to ensure the design met timing requirements.

## **Introduction**

Digital counters are fundamental sequential circuits used in timers, frequency dividers, digital clocks, and event counting applications. This project focuses on implementing a 4-bit synchronous up-counter and validating its functionality through RTL simulation, synthesis, and gate-level simulation.

## **Objectives**

- Design a 4-bit synchronous counter using Verilog HDL.
- Perform RTL simulation to verify functionality.
- Synthesize the RTL design into a gate-level netlist using Vivado.
- Perform gate-level simulation.
- Analyze timing reports and verify timing constraints.

## **Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

### **Software and Tools Used**

| <b>Tool</b>      | <b>Purpose</b>                       |
|------------------|--------------------------------------|
| Xilinx Vivado    | Design Entry, Synthesis, Simulation  |
| Verilog HDL      | Hardware Description Language        |
| Vivado Simulator | Functional and Gate-Level Simulation |

### **Design Description**

The 4-bit counter increments its output value on every positive edge of the clock signal. When the reset signal is asserted, the counter returns to zero.

#### **Inputs**

- clk : Clock signal
- rst : Reset signal

#### **Output**

- q[3:0] : 4-bit counter output

## **Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

### **RTL Code**

```
module counter(  
    input clk,  
    input rst,  
    output reg [3:0] q  
);  
always @(posedge clk or posedge rst)  
begin  
    if(rst)  
        q <= 4'b0000;  
    else  
        q <= q + 1'b1;  
    end  
end  
endmodule
```

### **Testbench**

```
`timescale 1ns/1ps  
module tb_counter;  
    reg clk;  
    reg rst;  
    wire [3:0] q;
```

**Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

```
counter uut(  
    .clk(clk),  
    .rst(rst),  
    .q(q)  
);  
always #5 clk = ~clk;  
initial begin  
    clk = 0;  
    rst = 1;  
    #20 rst = 0;  
    #100;  
    $finish;  
end  
endmodule
```

**Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

## **RTL Simulation**

Behavioral simulation was performed using the Vivado Simulator.

### **Expected Counter Sequence**

Clock Cycle Counter Value

|   |      |
|---|------|
| 0 | 0000 |
| 1 | 0001 |
| 2 | 0010 |
| 3 | 0011 |
| 4 | 0100 |
| 5 | 0101 |

The simulation confirmed correct counting behavior.

## **Synthesis**

The RTL design was synthesized using Vivado.

### **Synthesis Steps**

1. Create Vivado Project.
2. Add RTL source file.
3. Add Testbench.
4. Run Synthesis.
5. Open Synthesized Design.

## **Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

6. Generate Gate-Level Netlist.

### **Netlist Generation Command**

```
write_verilog -mode funcsim counter_netlist.v
```

The generated file counter\_netlist.v contains the synthesized gate-level representation of the design.

## **Gate-Level Simulation**

Post-synthesis functional simulation was performed using the synthesized netlist.

### **Procedure**

1. Run Post-Synthesis Functional Simulation.
2. Load gate-level netlist.
3. Execute simulation.
4. Compare output with RTL simulation.

### **Results**

The gate-level simulation matched the RTL simulation results, confirming successful synthesis.

## **Static Timing Analysis (STA)**

A timing constraint was applied using an XDC file.

### **Clock Constraint**

```
create_clock -period 20 [get_ports clk]
```

## **Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

Clock Frequency:

Frequency = 1 / Period

Frequency = 1 / 20 ns = 50 MHz

### **Timing Summary**

| Parameter    | Value    |
|--------------|----------|
| Clock Period | 20 ns    |
| WNS          | Positive |
| TNS          | 0 ns     |

Setup Violations 0

Hold Violations 0

The design met all timing constraints successfully.

## **Results**

- RTL simulation completed successfully.
- Counter output incremented correctly.
- Synthesis generated a valid gate-level netlist.
- Gate-level simulation matched RTL behavior.
- Timing constraints were satisfied.
- No setup or hold violations were detected.

## **Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

### **Applications**

- Digital clocks
- Event counters
- Frequency dividers
- Timers
- Embedded systems

### **Conclusion**

A 4-bit synchronous counter was successfully designed using Verilog HDL and implemented in Xilinx Vivado. RTL simulation verified the functionality of the design, while synthesis generated a gate-level netlist for implementation. Post-synthesis gate-level simulation confirmed the correctness of the synthesized design. Static Timing Analysis showed positive slack with no setup or hold violations, demonstrating that the design satisfies the specified timing requirements.

### **References**

1. S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Prentice Hall, 2003.
2. M. Morris Mano and Michael D. Ciletti, *Digital Design*, 6th Edition, Pearson Education, 2018.

**Synthesize RTL to a gate-level netlist using an academic flow or open-source tool and perform gate-level simulation**

3. Douglas J. Smith, *HDL Chip Design: A Practical Guide for Designing, Synthesizing and Simulating ASICs and FPGAs Using VHDL or Verilog*, Doone Publications, 1998.
4. Xilinx, *Vivado Design Suite User Guide: Synthesis (UG901)*.
5. Xilinx, *Vivado Design Suite User Guide: Logic Simulation (UG900)*.
6. Xilinx, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)*.
7. IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005.
8. Xilinx Documentation and Tutorials:  
<https://www.xilinx.com/support/documentation.html>
9. Verilog HDL Reference Guide:  
<https://www.verilog.com>
10. FPGA4Student, "Verilog Counter Design Examples":  
<https://www.fpga4student.com>
11. NPTEL Course: Digital System Design with Verilog:  
<https://nptel.ac.in>
12. Xilinx Vivado Design Suite Official Website:  
<https://www.xilinx.com/products/design-tools/vivado.html>